

PartitionedVdb — Usage Guide

A time-partitioned, tag-grouped vector database built on HNSW.

Vectors are bucketed by timestamp granularity (hourly → yearly) and custom partition tags, enabling efficient scoped searches across massive datasets.

Loading the Module

Option 1: Direct XLang Import (standalone scripts)

```
import xlang
quanta = xlang.importModule("quanta", fromPath="Quanta")
```

Option 2: Via Cantor Service (recommended for production)

When Cantor is running, load the module through it. This enables **singleton sharing** — the module is loaded once and shared across all processes.

```
import xlang

# Connect to Cantor via LRPC
cantor = xlang.importModule("cantor", thru="lrpc:1000")

# First process: Load the module into Cantor
quanta = cantor.LoadModule("quanta", "Quanta")

# Other processes: Query the already-loaded module (no reload, same instance)
quanta = cantor.QueryModule("quanta")
```

Why use Cantor?

- **Singleton:** `LoadModule` loads the native library once; `QueryModule` returns the same instance — no duplicate memory
- **Multi-process safe:** Multiple Python processes share the same engine via LRPC
- **Service lifecycle:** Module stays loaded as long as Cantor runs, independent of any Python process

Creating a VDB Instance

Once you have `quanta` (from either method above):

```

import numpy as np
import time

# Create a new VDB
vdb = quanta.partitioned_vdb(
    prefix="my_vdb",           # file prefix for saved data
    path="./data/vectors",     # storage directory
    dim=512,                   # embedding dimension
    granularity="monthly"      # time partition: hourly|daily|weekly|monthly|yearly
)

```

Core API

AddVectors(id, embedding, **kwargs)

Add one or more vectors with metadata.

Parameter	Type	Description
<code>id</code>	int or list	External ID(s) — single int for one vector, list for batch
<code>embedding</code>	numpy array	Float32 array, shape <code>(dim,)</code> for single or <code>(n*dim,)</code> for batch
<code>timestamp</code>	int (kwarg)	Millisecond timestamp (e.g. <code>int(time.time()*1000)</code>)
<code>partition</code>	str (kwarg)	Custom partition tag (default: <code>"default"</code>)
<code>chunks</code>	str or list (kwarg)	Text label(s) stored with each vector
<code>num_threads</code>	int (kwarg)	Thread count for HNSW insertion (-1 = auto)

```

now_ms = int(time.time() * 1000)

# Single vector
emb = np.random.randn(512).astype(np.float32)
emb /= np.linalg.norm(emb)
vdb.AddVectors(1, emb, timestamp=now_ms, partition="cam_01", chunks="frame_001.jpg")

# Batch: 3 vectors with sequential IDs starting at 100
batch = np.random.randn(3, 512).astype(np.float32)
batch = (batch / np.linalg.norm(batch, axis=1, keepdims=True)).flatten()
vdb.AddVectors(100, batch, timestamp=now_ms, partition="cam_02", chunks=
["a.jpg", "b.jpg", "c.jpg"])

```

Lookup(query, topK, **kwargs)

Search for the most similar vectors. Returns a list of 5-element tuples.

Parameter	Type	Description
<code>query</code>	numpy array	Query embedding, shape (dim,)
<code>topK</code>	int	Max results to return
<code>partition</code>	str (kwarg)	Filter to a single partition tag
<code>partitions</code>	list (kwarg)	Filter to multiple partition tags
<code>ts_start</code>	int (kwarg)	Start of time range filter (ms, inclusive)
<code>ts_end</code>	int (kwarg)	End of time range filter (ms, inclusive)
<code>dedup</code>	float (kwarg)	Cosine similarity threshold for deduplication (0-1)

Returns: list of [id, score, chunk_text, partition_key, timestamp_ms]

```
query = np.random.randn(512).astype(np.float32)
query /= np.linalg.norm(query)

# Basic search
results = vdb.Lookup(query, 10)
for r in results:
    print(f"id={r[0]}, score={r[1]:.4f}, chunk={r[2]}, partition={r[3]}, ts={r[4]}")

# Filter by partition
results = vdb.Lookup(query, 10, partition="cam_01")

# Filter by multiple partitions
results = vdb.Lookup(query, 10, partitions=["cam_01", "cam_02"])

# Precise time range (milliseconds)
one_hour_ago = now_ms - 3600 * 1000
results = vdb.Lookup(query, 10, ts_start=one_hour_ago, ts_end=now_ms)

# Combined: partition + time range
results = vdb.Lookup(query, 10, partition="cam_01", ts_start=one_hour_ago, ts_end=now_ms)

# Deduplication: remove results with cosine sim > 0.95
results = vdb.Lookup(query, 10, dedup=0.95)
```

Save(path) / Load(path)

Persist and restore the entire VDB (index + metadata + timestamps).

```
# Save (empty string = use current basePath)
vdb.Save("")

# Save to a specific path
vdb.Save("/backup/vectors")

# Load into a new instance
vdb2 = quanta.partitioned_vdb(prefix="my_vdb", path="./data/vectors", dim=512)
vdb2.Load("./data/vectors")

# Now vdb2 has all the data, including timestamps
results = vdb2.Lookup(query, 5)
```

Files created on disk:

```
./data/vectors/
├─ my_vdb_manifest.db          # SQLite: config + partition registry
├─ my_vdb_2026-03_0.hnsw      # HNSW index (time=2026-03, partition=default)
├─ my_vdb_2026-03_0.vdb      # Labels + chunks + timestamps
├─ my_vdb_2026-03_1.hnsw      # HNSW index (time=2026-03, partition=cam_01)
└─ my_vdb_2026-03_1.vdb      # Labels + chunks + timestamps
```

ListPartitions()

List all partitions (on-disk and in-memory).

```
for p in vdb.ListPartitions():
    print(f"key={p['key']}, ts={p['ts_partition']}, tags={p['tags']}, loaded={p['loaded']}")
```

AddPartitionTag(index, tag) / GetPartitionInfo(tag)

Manage custom partition tags.

```
# Add an alias tag to partition index 1
vdb.AddPartitionTag(1, "entrance_camera")

# Query partition info
info = vdb.GetPartitionInfo("cam_01")
print(f"index={info['index']}, tags={info['tags']}")
```

QueryLabelByID(id, **kwargs)

Look up the stored chunk text for a specific vector ID.

```
# Search all partitions
label = vdb.QueryLabelByID(42)

# Narrow search to a specific partition + timestamp
label = vdb.QueryLabelByID(42, partition="cam_01", timestamp=now_ms)
```

Grouping(items, threshold, **kwargs)

Cluster vectors by centroid similarity. Input is a list of dicts with IDs.

```
items = [
    {"image_id": 1, "timestamp": ts1, "device_id": "cam_01"},
    {"image_id": 2, "timestamp": ts2, "device_id": "cam_01"},
    {"image_id": 3, "timestamp": ts3, "device_id": "cam_02"},
]

# Group items with cosine similarity >= 0.85
groups = vdb.Grouping(items, 0.85,
    id_key="image_id",
    partition_key="device_id",
    timestamp_key="timestamp"
)

# Returns: [[1, 2], [3]] - each sublist is a group of similar IDs
```

Singleton Access via Cantor LRPC

For multi-process deployments, load the module once via Cantor and share it:

```
import xlang

# Process 1: Load the module
cantor = xlang.importModule("cantor", thru="lrpc:1000")
quanta = cantor.LoadModule("quanta", "Quanta")
vdb = quanta.partitioned_vdb(prefix="shared", path="/data/vdb", dim=512)

# Process 2+: Query the already-loaded module (singleton)
cantor = xlang.importModule("cantor", thru="lrpc:1000")
quanta = cantor.QueryModule("quanta") # same instance as Process 1
vdb = quanta.partitioned_vdb(prefix="shared", path="/data/vdb", dim=512)
vdb.Load("/data/vdb")
```

CLIP Image Embedding Example

```

import torch, clip
from PIL import Image

device = "cuda" if torch.cuda.is_available() else "cpu"
model, preprocess = clip.load("ViT-B/32", device=device)

def image_to_embedding(image_path):
    img = Image.open(image_path).convert("RGB")
    img_input = preprocess(img).unsqueeze(0).to(device)
    with torch.no_grad():
        emb = model.encode_image(img_input)
        emb = emb / emb.norm(dim=-1, keepdim=True)
    return emb.cpu().numpy().flatten().astype(np.float32)

def text_to_embedding(text):
    text_input = clip.tokenize([text]).to(device)
    with torch.no_grad():
        emb = model.encode_text(text_input)
        emb = emb / emb.norm(dim=-1, keepdim=True)
    return emb.cpu().numpy().flatten().astype(np.float32)

# Index images
vdb = quanta.partitioned_vdb(prefix="images", path="./image_vdb", dim=512)
for i, path in enumerate(image_paths):
    emb = image_to_embedding(path)
    vdb.AddVectors(i, emb,
        timestamp=int(os.path.getmtime(path) * 1000),
        chunks=os.path.basename(path)
    )
vdb.Save("")

# Search by text
results = vdb.Lookup(text_to_embedding("a red car"), 5)

# Search by image similarity
results = vdb.Lookup(image_to_embedding("query.jpg"), 5)

# Search within last 24 hours only
now = int(time.time() * 1000)
results = vdb.Lookup(text_to_embedding("person"), 5,
    ts_start=now - 86400000, ts_end=now)

```

Constructor Parameters

Parameter	Default	Description
<code>prefix</code>	<code>"vdb"</code>	Filename prefix for all persisted files
<code>path</code>	<code> "." </code>	Base directory for storage
<code>dim / dimension</code>	<code>512</code>	Embedding vector dimension

Parameter	Default	Description
<code>granularity</code>	<code>"monthly"</code>	Time bucketing: <code>hourly</code> , <code>daily</code> , <code>weekly</code> , <code>monthly</code> , <code>yearly</code>
<code>space</code>	<code>"l2"</code>	Distance metric: <code>l2</code> , <code>ip</code> (inner product), <code>cosine</code>
<code>max_elements</code>	<code>100000</code>	Max vectors per HNSW partition
<code>M</code>	<code>16</code>	HNSW graph connectivity
<code>ef_construction</code>	<code>200</code>	HNSW build-time search depth
<code>ef_search</code>	<code>50</code>	HNSW query-time search depth

Time-Range Filtering: How It Works

Filtering operates at **two levels** for performance:

1. **Partition pre-filter** — Entire time partitions (e.g. months) outside the range are skipped entirely. Zero cost for excluded partitions.
2. **Per-record post-filter** — Within matching partitions, each result's stored millisecond timestamp is checked against `[ts_start, ts_end]`. Only exact matches are returned.

This two-level approach gives you both the efficiency of coarse partitioning and the precision of exact millisecond filtering.